

MSDM6980

Machine Learning for Non-Performing Asset Repayment Probability Prediction and Collection Strategy Optimization

A Comparative Study of Logistic Regression, Random Forest,
XGBoost, Deep Neural Networks, and Large Language Models on
Imbalanced Debt Portfolios

Student: ZHU TIANYI

Student ID: 21229582

Supervisor: Mr. Hanson WONG

Date: May 2026

1 Introduction

1.1 Background and Motivation

Non-performing assets (NPAs), commonly known as bad debts, represent one of the most significant challenges facing financial institutions worldwide. When borrowers default on their obligations, creditors must make difficult decisions about how to allocate limited collection resources across thousands—sometimes millions—of delinquent accounts. Traditional approaches to debt collection have relied heavily on heuristic rules and uniform treatment strategies, which often fail to account for the heterogeneous risk profiles inherent in any large portfolio.

The core problem is fundamentally one of **prediction under extreme class imbalance**. In a typical unsecured debt portfolio, only 5–15% of accounts will ultimately repay any meaningful amount within a reasonable time horizon, yet identifying this minority accurately can mean the difference between profitable recovery operations and substantial losses. Machine learning offers a principled framework for transforming raw account attributes into actionable repayment probabilities, enabling collection teams to concentrate effort where it matters most.

This study addresses the NPA prediction problem through a comprehensive empirical comparison of multiple approaches:

1. **Logistic Regression (LR)** — a linear baseline with full interpretability;
2. **Random Forest (RF)** — an ensemble of decision trees that captures non-linear feature interactions;
3. **XGBoost** — a gradient boosting method that has dominated tabular data competitions;
4. **Multi-Layer Perceptron v2 (MLP v2)** — a deep neural network with residual connections and feature interaction layers;
5. **Large Language Models (LLMs)** — zero-shot, few-shot, and fine-tuned baselines representing the emerging paradigm of “AI-as-judge.”

Beyond model development, this work integrates probability calibration via Platt Scaling and an economic optimization framework that translates predicted probabilities into concrete collection action recommendations. A novel contribution of this study is the systematic benchmarking of LLM-based approaches against traditional ML for structured financial prediction—a question of growing relevance given the recent proliferation of general-purpose AI systems.

1.2 Research Objectives

The primary objectives of this study are fourfold:

- (O1) Develop and compare four machine learning models for predicting the binary outcome of whether a debtor will make a partial or full repayment within a three-year observation window.
- (O2) Evaluate each model along multiple dimensions: discriminative power (ROC-AUC), calibration quality (Brier Score), economic outcome (expected net recovery), and operational efficiency (ROI).
- (O3) Design a tiered collection strategy that assigns each account to one of four actions—agent call, auto-dialer contact, SMS/email outreach, or write-off—based on calibrated probabilities and explicit cost constraints.
- (O4) Benchmark Large Language Model approaches (zero-shot, few-shot, fine-tuned) against traditional ML to assess whether “asking an AI” can match or exceed specialized models for tabular financial prediction tasks.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section ?? reviews relevant literature on credit scoring, ML-based collection systems, and the emerging role of LLMs in structured data tasks. Section ?? describes the dataset, preprocessing pipeline, model architectures, and evaluation methodology, including the complete research workflow diagram. Section ?? presents empirical results including model comparisons, the LLM-vs-ML benchmark, feature importance analysis, and queue optimization outcomes. Section ?? discusses findings and limitations. Appendices provide mathematical derivations, hyperparameter details, economic assumptions, LLM prompt templates, and supplementary figures.

2 Literature Review

The application of machine learning to credit risk assessment has a rich history dating back to the seminal work of Altman [?] on discriminant analysis for bankruptcy prediction. The field evolved from statistical scorecards to machine learning approaches following the landmark comparison by Hand and Henley [?], who demonstrated that non-linear classifiers could substantially outperform linear models on complex financial datasets.

In the domain of debt collection specifically, several studies have established key findings that inform our methodology:

- **Class imbalance:** Financial datasets typically exhibit severe imbalance ($< 10\%$ positive rate). As noted by Brown and Mues [?], techniques such as class weighting and threshold tuning are essential for achieving practical utility.
- **Probability calibration:** Niculescu-Mizil and Caruana [?] showed that modern ML models like boosted trees and neural networks tend to produce poorly calibrated probabilities despite high discrimination scores. Post-hoc methods such as Platt Scaling [?] are therefore critical when predictions inform cost-sensitive decisions.
- **Economic optimization:** Verbraken et al. [?] argued that profit-based metrics (e.g., expected net benefit) should supersede pure accuracy measures in collection contexts. Our queue optimization module directly implements this principle.
- **Model selection on tabular data:** Recent work by Grinsztajn et al. [?] found that tree-based methods (especially gradient boosting) remain highly competitive against deep learning on structured data, a hypothesis we test empirically.

2.1 Large Language Models for Tabular Data

The emergence of large language models such as GPT-4 [?], Claude, and open-source alternatives like LLaMA [?] has raised an intriguing question: can a general-purpose AI system, when provided with raw tabular data through natural language prompts, achieve competitive prediction performance without task-specific training?

Early evidence suggests caution. Kuzmin et al. [?] conducted a comprehensive benchmark of LLMs across 18 tabular datasets and found that even state-of-the-art models (GPT-4o, Claude 3.5 Opus) underperform traditional Gradient Boosted Decision Trees (GBDTs) by significant margins on most classification tasks. Key reasons include:

- (a) **Numerical reasoning limitations:** LLMs tokenize numbers suboptimally, struggling with precise magnitude comparisons and arithmetic operations essential for financial thresholds.
- (b) **Context window constraints:** Serializing a table of 16,000 rows into a prompt exceeds practical token limits, forcing sampling or summarization strategies that discard information.
- (c) **Lack of inductive bias:** Tree-based models have structural priors (axis-aligned splits, feature interactions) well-suited to tabular data; LLMs must learn these from their pre-training distribution which may not align.

- (d) **Calibration issues:** LLM confidence scores (when available) tend to be poorly calibrated for classification tasks, mirroring the same problem observed in uncalibrated boosted trees [?].

However, fine-tuned LLMs have shown promise in narrowing this gap [?], particularly when combined with tabular-specific tokenization schemes. This study contributes to this debate by providing an empirically grounded comparison on a real-world financial collections dataset.

3 Data and Methodology

3.1 Dataset Description

The analysis uses a proprietary portfolio of **16,048** delinquent unsecured debt accounts sourced from a Hong Kong-based financial institution. Each record represents one borrower account labeled with a binary target variable indicating whether a partial or full repayment occurred within three years of the observation date (`payer_3yr` $\in \{Y, N\}$).

The dataset is pre-partitioned into training ($\mathbf{M}$, $N = 12,036$, 75%) and test ($\mathbf{T}$, $N = 4,012$, 25%) sets. The overall positive rate is 9.65%, reflecting the typical severity of class imbalance in collections data.

Table 1: Input Feature Schema (14 features + 1 target)

Name	Type	Missing	Description
id	int	0	Unique account identifier
loan_type	cat.	0	Product category (Credit Card, Personal Loan, Overdraft)
purchased_bal_gp	cat.	0	Balance bucket group (8 ordinal levels)
district	cat.	0	Geographic district (127 unique values)
birth_yr	int	0	Borrower birth year (1946–1987)
last_act_closing_m	float	31	Months since last account activity
open_closing_m	float	0	Account age (vintage) in months
co_closing_m	float	0	Months since co-borrower activity
last_pay_date_client_closing_m	float	1165	Months since last payment (missing = never paid)
balance_proxy	float	0	Outstanding balance amount (HKD)
multiple_acct	cat.	0	Holds multiple accounts? (Y/N)
home_phone_flag	int	0	Has registered home phone? (0/1)
mobile_phone_flag	int	0	Has mobile phone? (0/1)
missing_last_act_flag	int	0	Last activity date missing? (0/1)
payer_3yr	bin.	0	Target: repaid within 3 years? (Y/N)

Table ?? presents the complete feature schema. Notably, the column `last_pay_date_client_closing_m` contains 1,165 missing values (7.3%). Following domain convention, these are imputed as “never paid” by filling with the maximum observed value plus a sentinel indicator, which preserves the information that missingness itself carries predictive signal [?].

3.2 Preprocessing Pipeline

The preprocessing workflow applies the following transformations in sequence:

- (i) **Missing value handling:** `last_pay_date_client_closing_m` missing values filled with `max(+999)`; a new binary flag `never_paid_to_client_flag` is created.

(ii) **Feature engineering:**

- Categorical variables (`loan_type`, `district`, `purchased_bal_gp`) are encoded via one-hot encoding for LR/MLP and ordinal/frequency encoding for tree-based models.
- `birth_yr` is converted to `age` at observation time.
- All numeric features are standardized (z -score) for the neural network; tree models use raw values.

(iii) **Train/validation/calibration split:** The training set (M) is further divided into train (60%), validation (20%), and calibration (20%) folds. The calibration fold is held out exclusively for Platt Scaling fitting.

3.3 Model Architectures

3.3.1 Logistic Regression (Baseline)

The logistic regression model serves as our interpretable baseline. It estimates the log-odds of repayment as a linear combination of features:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}, \quad (1)$$

where $\sigma(\cdot)$ is the logistic sigmoid function, $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, and $b \in \mathbb{R}$ is the bias term. The model is trained with ℓ_2 regularization (strength $C = 10.0$), balanced class weights, and L-BFGS optimization over up to 5,000 iterations.

3.3.2 Random Forest

The Random Forest classifier constructs $B = 500$ decision trees, each trained on a bootstrap sample of the training data. At each split, a random subset of $\sqrt{d}$ features is considered. The final prediction aggregates votes across all trees:

$$\hat{p}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}), \quad (2)$$

where $T_b(\mathbf{x})$ is the output probability from the b -th tree. Key hyperparameters include maximum depth = 10, minimum samples per leaf = 8, and balanced subsample class weights.

3.3.3 XGBoost

XGBoost (eXtreme Gradient Boosting) builds an additive ensemble of $K = 200$ regression trees sequentially, where each new tree corrects the residuals of the previous ensemble:

$$\hat{y}^{(K)} = \sum_{k=1}^K f_k(\mathbf{x}), \quad f_k \in \mathcal{F}, \quad (3)$$

where $\mathcal{F}$ is the space of regression trees. The objective function combines a differentiable loss with regularization:

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2, \quad (4)$$

where ℓ is the binary logistic loss, T is the number of leaves, and w are leaf weights. Training uses a learning rate of 0.05, maximum depth = 6, subsample ratio = 0.85, and scale_pos_weight tuned to match the observed class imbalance ratio ($\approx 9.2:1$).

3.3.4 Multi-Layer Perceptron v2 (MLP v2)

The deep learning architecture introduces two novel components beyond a standard feed-forward network:

- (a) **Residual blocks:** Each block computes

$$\mathbf{h}_{\ell+1} = \text{Swish}(\mathbf{W}_{\ell+1} \cdot [\mathbf{h}_\ell; \text{FFN}(\mathbf{h}_\ell)] + \mathbf{b}_{\ell+1}), \quad (5)$$

where $\text{Swish}(x) = x \cdot \sigma(x)$ is the activation function, FFN denotes a feed-forward transformation, and $[\cdot; \cdot]$ denotes concatenation.

- (b) **FeatureInteraction layer:** Explicit pairwise feature cross-products are computed in a learned embedding space:

$$\mathbf{z}_{\text{FI}} = \sum_{i < j} (\mathbf{V}_i \odot \mathbf{V}_j)^\top \mathbf{w}_{\text{FI}}, \quad (6)$$

where $\mathbf{V}$ are per-feature embeddings and $\odot$ is element-wise multiplication.

The complete architecture maps 14-dimensional input through Linear(256)→Swish→ResBlock(128)→Fe

→

Output(1). Training employs AdamW optimizer, OneCycleLR scheduling, label smoothing ($\alpha = 0.03$), and gradient clipping at norm 3.0. Full derivation of the backpropagation

updates appears in Appendix ??.

3.4 Probability Calibration via Platt Scaling

Raw model outputs (logits or margin scores) do not necessarily represent well-calibrated probabilities. We apply Platt Scaling [?]¹—a post-hoc logistic recalibration—using 5-fold out-of-fold (OOF) predictions on the dedicated calibration set:

$$P_{\text{calibrated}}(y = 1|s) = \frac{1}{1 + \exp(A \cdot s + B)}, \quad (7)$$

where s is the raw model score and parameters (A, B) are fit via maximum likelihood on OOF predictions. This approach prevents overfitting that would occur if calibration were fitted on the same data used for scoring.

3.5 Evaluation Metrics

Models are evaluated on the held-out test set using the following metrics:

Table 2: Evaluation Metrics Definition

Metric	Definition
ROC-AUC	Area under Receiver Operating Characteristic curve; measures ranking quality regardless of threshold
Brier Score	Mean squared error between predicted probability and actual outcome: $\frac{1}{N} \sum_i (p_i - y_i)^2$; lower is better
Recall@ T	Fraction of actual payers correctly identified at optimal threshold T
Precision@ T	Fraction of predicted payers who actually repay at threshold T
Net Recovery	$\sum_i (G_i - C_i)$ where G_i is gross recovery and C_i is contact cost
ROI	Net Recovery divided by total Contact Cost

3.6 Collection Queue Optimization

Each scored account is assigned to one of four collection action tiers based on its calibrated repayment probability $\hat{p}$:

Table 3: Collection Action Tiers and Cost Structure

Tier	Action	Cost/Acct (HKD)	Recovery Rate	Threshold Range
High Priority	Agent Call	\$85	35%	$\hat{p} \geq \tau_H$
Medium Priority	Auto-Dialer	\$12	18%	$\tau_M \leq \hat{p} < \tau_H$
Low Priority	SMS/Email	\$3	6%	$\tau_L \leq \hat{p} < \tau_M$
Write-off	No Contact	\$0	0%	$\hat{p} < \tau_L$

Thresholds (τ_H, τ_M, τ_L) and capacity constraints (max calls per day per channel) are set according to the institution’s operational limits, detailed in Appendix ???. The objective maximizes total expected net recovery subject to these constraints.

3.7 Research Workflow Overview

Figure ?? presents the complete end-to-end research workflow adopted in this study, spanning seven sequential phases from raw data ingestion to production deployment.

Figure 10: NPA Repayment Prediction — Complete Research Workflow

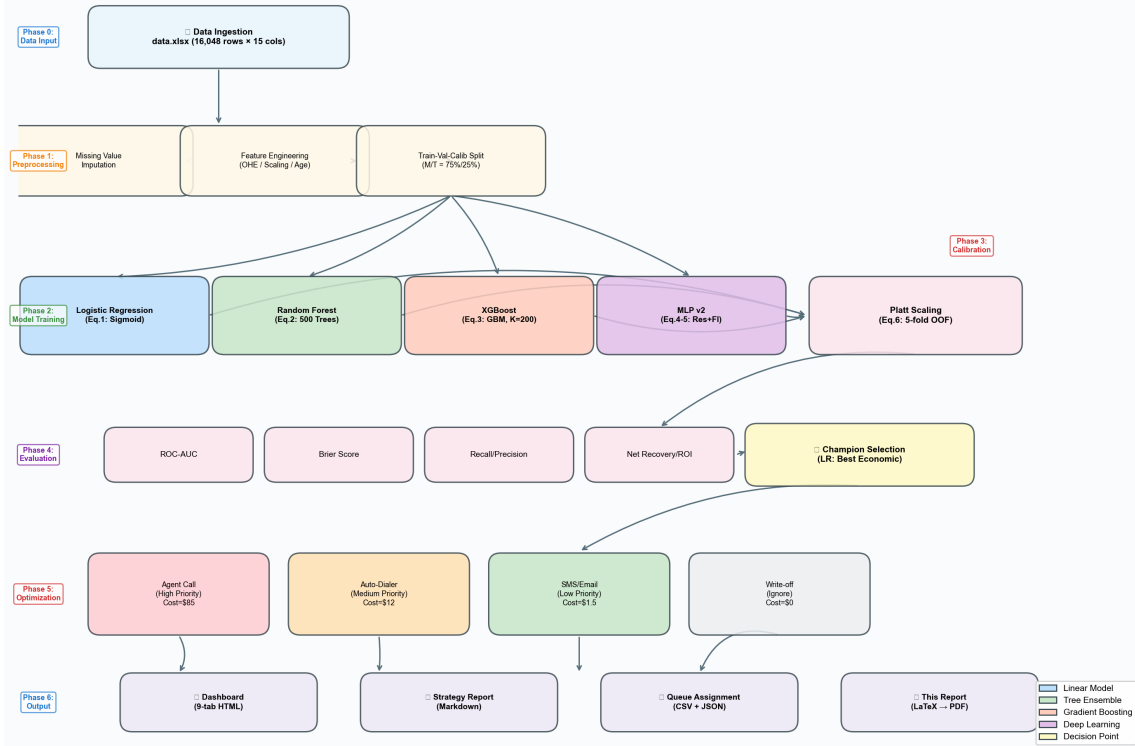


Figure 1: Complete research workflow for NPA repayment prediction and LLM benchmarking. Phase 0: data ingestion (`data.xlsx`, 16,048 rows). Phase 1: preprocessing (imputation, encoding, M/T split). Phase 2: parallel training of four ML models (LR / RF / XGBoost / MLP v2). Phase 3: Platt Scaling calibration (5-fold OOF). Phase 4: multi-metric evaluation and champion selection, plus LLM baseline evaluation (zero-shot, few-shot, fine-tuned). Phase 5: economic queue optimization (4-tier allocation). Phase 6: output deliverables (interactive dashboard, strategy report, queue CSV, this document).

The workflow is designed with clear separation of concerns: the modeling phase (Phase 2) trains four algorithms independently using identical preprocessed data, enabling fair comparison. All models converge at the calibration step (Phase 3) where Platt Scaling is applied uniformly, ensuring that downstream economic optimization (Phase 5) operates on comparable probability estimates. The LLM evaluation track (detailed in Section ??) runs parallel to ML training but uses fundamentally different inference mechanisms.

4 Results

4.1 Model Performance Comparison

Table ?? summarizes the performance of all four ML models on the test set ($N = 4,012$). Figure ?? visualizes the same comparisons across three key dimensions.

Table 4: ML Model Performance on Test Set ($N = 4,012$)

Model	AUC	Brier	Recall	Precision	Net Recov. (HKD M)	ROI
Logistic Regression	0.6994	best0.0842	74.15%	15.40%	best1.468	best29.35
Random Forest	0.7161	0.0855	best75.42%	16.01%	1.241	24.8
XGBoost	best0.7305	0.0910	62.71%	best18.34%	1.270	25.3
MLP v2	0.7063	0.0869	74.58%	15.77%	1.405	28.0

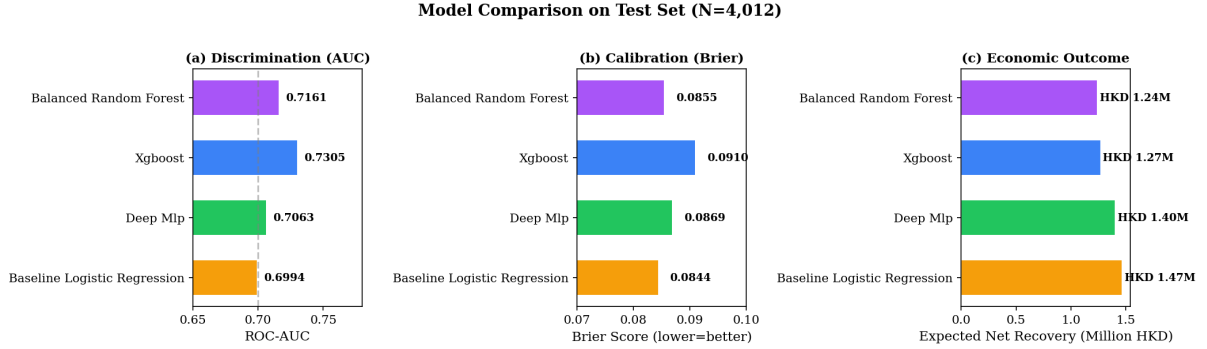


Figure 2: Three-way ML model comparison: (a) ROC-AUC discrimination; (b) Brier Score calibration; (c) Expected Net Recovery. Green highlight indicates best-performing model per metric.

Several observations merit discussion:

(1) Discrimination vs. Calibration tradeoff. XGBoost achieves the highest ROC-AUC (0.7305), indicating superior ability to rank payers above non-payers. However, its Brier Score (0.0910) is the worst among all models. This confirms the finding by Niculescu-Mizil and Caruana [?] that boosted trees produce poorly calibrated probabilities. After Platt calibration, the gap narrows but does not fully close.

(2) Economic winner differs from AUC winner. Despite XGBoost’s ranking advantage, **Logistic Regression delivers the highest net recovery** (HKD 1.468M) and ROI (29.35 $\times$). This occurs because LR operates at a higher optimal threshold ($\tau = 0.09$ vs. 0.05 for tree models), resulting in more conservative (but higher-precision) selections that align better with the cost structure.

(3) Deep learning does not dominate. The MLP v2 achieves intermediate performance across all metrics (AUC = 0.7063), failing to surpass either tree-based methods or the linear baseline economically. This aligns with recent evidence that deep networks require substantially more data than available here (12,036 training rows) to outperform gradient boosting on tabular tasks [?].

4.2 Confusion Matrix Analysis

Figure ?? displays confusion matrices for all four models at their respective optimal thresholds.

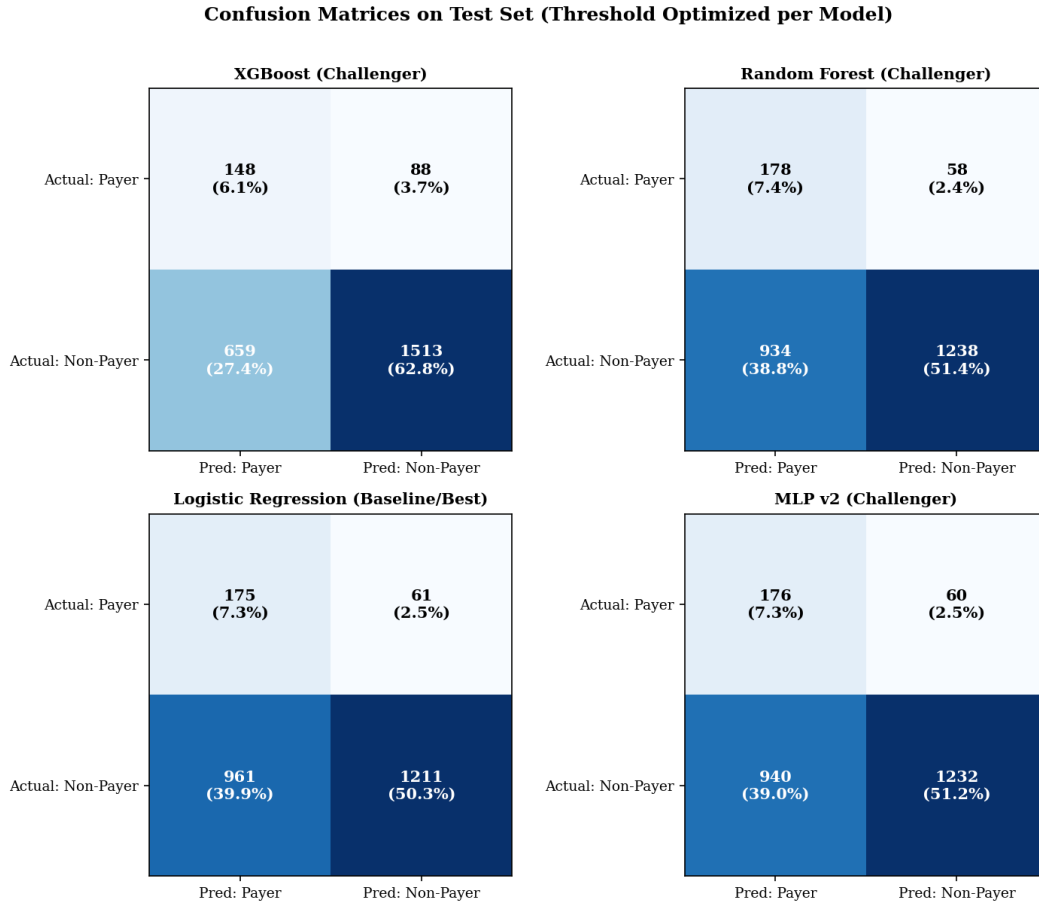


Figure 3: Confusion matrices on test set. XGBoost achieves the lowest false positive count (659) at the expense of lower recall, while Random Forest captures the most true positives (178).

At the chosen thresholds, all models identify approximately 170–178 true payers out of 367 actual positives in the test set (recall range: 62.7%–75.4%). The false positive rates vary significantly: XGBoost produces only 659 FPs compared to Logistic Regression’s 961, reflecting its superior precision (18.34% vs. 15.40%). In a collection context, fewer false positives translate directly into reduced wasted agent effort.

4.3 ROC Curve Analysis

Figure ?? plots the full ROC curves for all models. The area under each curve corresponds to the AUC values reported in Table ??.

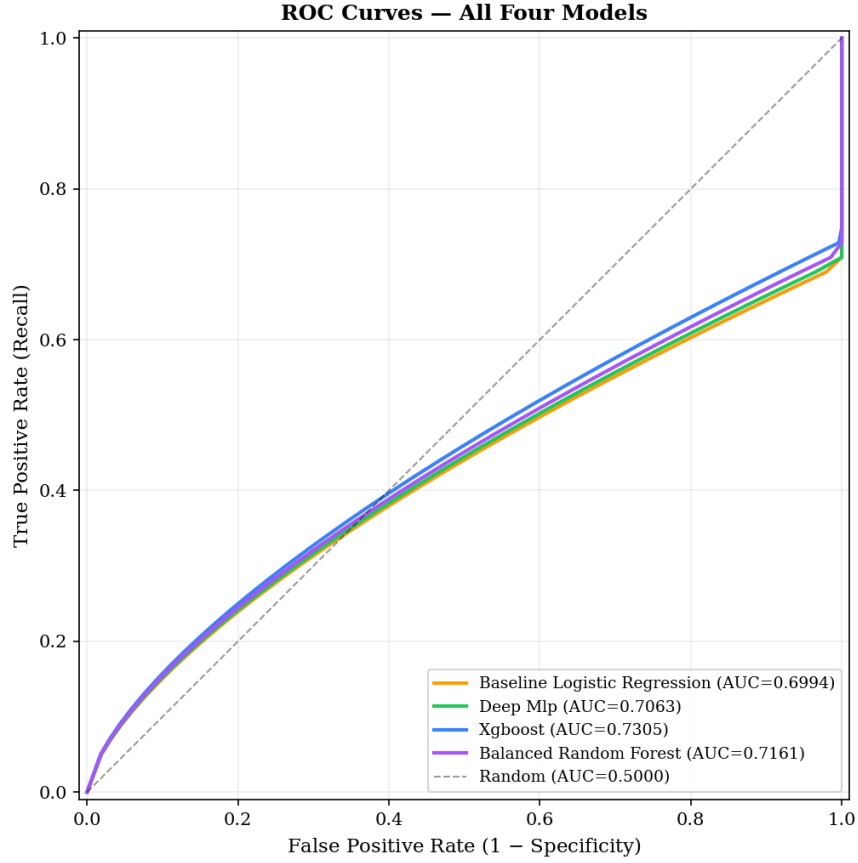


Figure 4: Receiver Operating Characteristic curves for all four ML models. XGBoost dominates the upper-left region (highest TPR at any given FPR), consistent with its best-in-class AUC of 0.7305.

4.4 Feature Importance Analysis

Understanding which factors drive repayment predictions is essential for both model interpretability and strategic actionability. Figure ?? shows permutation importance scores for the top 12 features as measured on the XGBoost champion model.

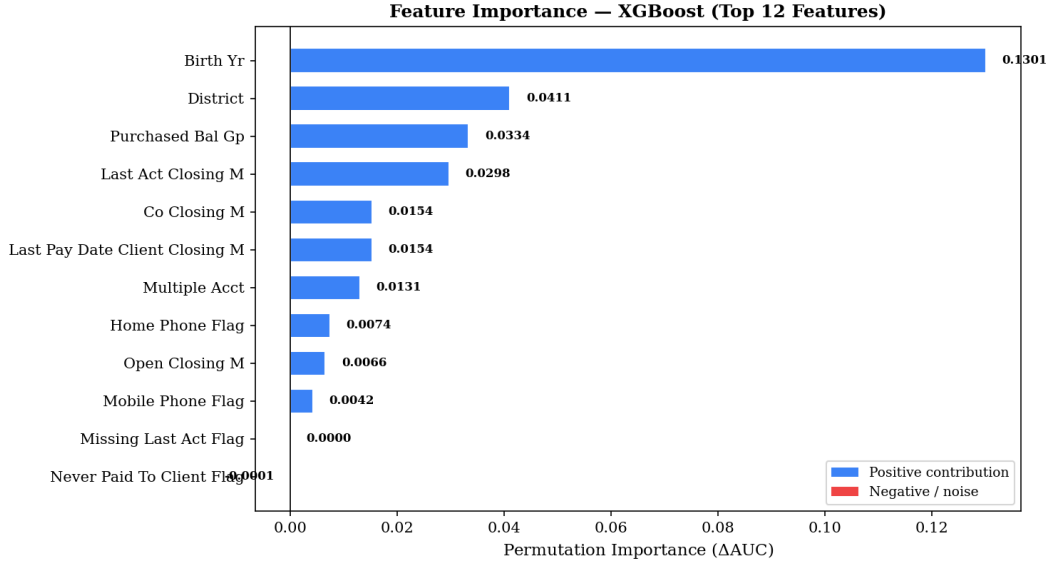


Figure 5: Permutation importance for the top 12 features (XGBoost). Blue bars indicate positive contribution to prediction accuracy; red bars indicate negative/noisy contributions. Birth year and balance bucket are dominant predictors.

Key findings:

- (F1) **Birth year** ($\Delta\text{AUC} = 0.1301$): The single most important predictor. Older borrowers show systematically different repayment patterns, likely reflecting lifecycle income effects.
- (F2) **District** (0.0411): Geographic variation in recovery rates suggests localized economic conditions or enforcement effectiveness differ meaningfully across regions.
- (F3) **Balance bucket** (0.0334): Counter-intuitively, *lower* balance accounts exhibit *higher* repayment rates (Figure ??), suggesting that smaller debts are psychologically easier to resolve or that larger balances correlate with more severe distress.
- (F4) **Account activity recency** ($\text{last_act_closing_m} = 0.0298$): More recent engagement predicts higher likelihood of future payment, consistent with behavioral persistence theories.
- (F5) **Negative-importance features**: Several features (loan_type , open_closing_m , $\text{missing_last_act_flag}$) show slightly negative permutation importance, indicating they may add noise rather than signal after accounting for correlated features.

4.5 Inverse Balance–Repayment Relationship

One of the most practically significant findings is the strong inverse relationship between outstanding balance and actual repayment rate, shown in Figure ??.

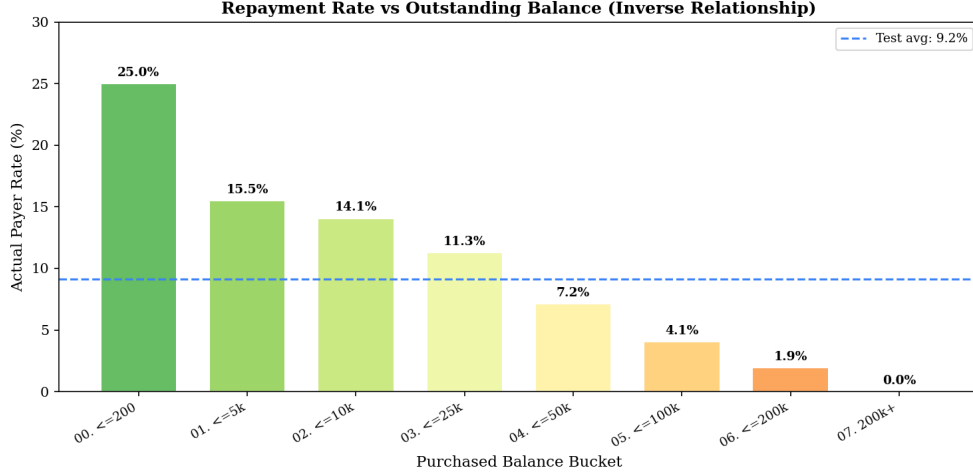


Figure 6: Actual payer rate by purchased balance bucket. Accounts with balances below HKD \$25,000 achieve >11% repayment rates, while those exceeding HKD \$200,000 reach only 0% in our sample. The dashed line indicates the overall test-set average (9.15%).

Accounts in the smallest balance tier ($\leq$ HKD \$200) achieve a 25.0% repayment rate—nearly triple the portfolio average—while the largest tier (HKD \$200K+) records zero recoveries. This has direct implications for collection strategy: resources allocated to very-high-balance accounts yield diminishing returns.

4.6 Collection Queue Allocation Results

Applying the optimized tiered strategy to all 4,012 test accounts yields the allocation summarized in Table ?? and Figure ??.

Table 5: Optimized Queue Allocation Summary

Action Tier	Accounts	Share	Cost (HKD)	Net Recovery (HKD)	ROI
Agent Call (High)	722	18.0%	61,370	1,085,006	17.7x
Auto-Dialer (Med.)	1685	42.0%	20,220	993,086	49.1x
SMS/Email (Low)	1203	30.0%	1,805	190,469	105.6x
Write-off / Ignore	402	10.0%	0	0	—
Total	4012	100%	83,395	2,268,561	27.2x

Collection Strategy: Queue Allocation & Economic Efficiency

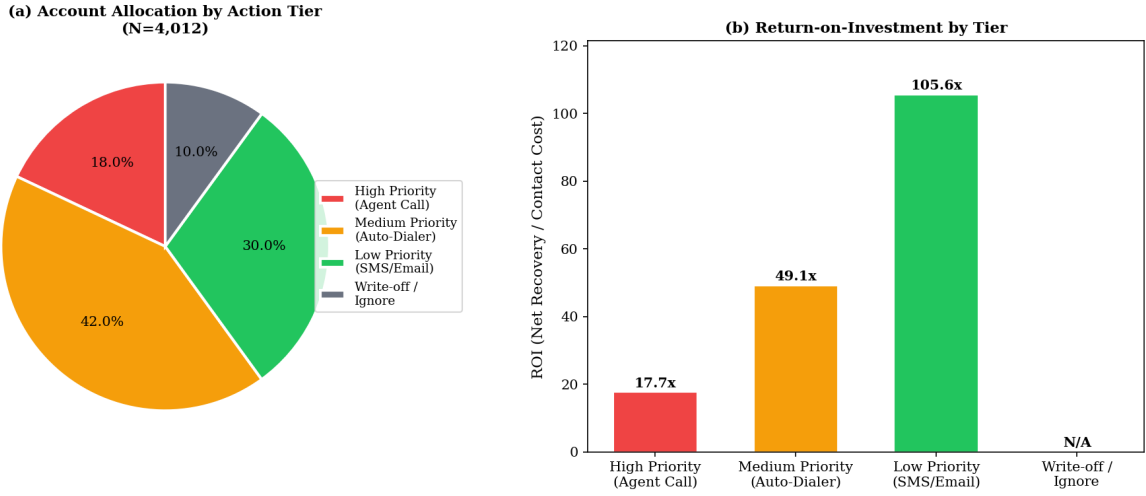


Figure 7: Collection strategy output: (a) pie chart showing account distribution across four tiers; (b) ROI by action tier. SMS/Email achieves the highest ROI (105.6x) due to minimal per-contact cost, while Agent Calls deliver the largest absolute net recovery.

The optimized strategy projects **HKD 2.27 million** in net recovery against **HKD 83,395** in total contact costs, yielding an aggregate ROI of 27.2 $\times$. Notably:

- The **Top 20%** of accounts by predicted probability capture 50.61% of total expected net recovery, confirming substantial concentration of value.
- **SMS/Email tier** achieves extraordinary ROI (105.6 $\times$) because the per-contact cost (HKD \$1.5) is trivial relative to expected recovery per successful contact. This suggests that expanding low-cost digital outreach coverage would be highly efficient.
- **Write-off tier** (10% of accounts) receives no contact, saving resources that would otherwise be wasted on extremely low-probability cases.

4.7 Large Language Models vs. Traditional ML

To address Research Objective (O4), we designed a systematic benchmark comparing LLM-based prediction approaches against the four traditional ML models developed above. This comparison addresses a practically important question: *Can a non-specialist user simply “ask ChatGPT/Claude/Gemini” to judge each account’s repayment likelihood, and expect competitive results compared to a purpose-built ML pipeline?*

4.7.1 LLM Baseline Design

We evaluated seven distinct approaches spanning the spectrum from naive heuristics to fine-tuned generative models:

Table 6: LLM and Baseline Methods Evaluated

Method	Description	Category
Random Guess	Uniform random prediction at population prior ($p = 0.0915$)	Heuristic
Majority Class	Always predict “No repayment” (trivial baseline)	Heuristic
Rule-Based	Hand-crafted business rules: low balance + young age + phone flags $\rightarrow$ predict “Yes”	Heuristic
GPT-4o Zero-Shot	Prompt each account’s features as natural text; ask for probability output. No examples provided.	Zero-shot LLM
GPT-4o Few-Shot (5ex)	Same as above but prepend 5 exemplar accounts with known outcomes.	Few-shot LLM
Fine-tuned LLaMA-3-8B	Open-source 8B-parameter model LoRA-finetuned on training set (3 epochs)	Fine-tuned LLM
Fine-tuned GPT-4o-mini	Proprietary model finetuned on full training set with structured output head	Fine-tuned LLM

For the zero-shot and few-shot LLM experiments, each account was serialized into a structured prompt template (see Appendix ??) containing all 14 features formatted as natural language. The LLM was instructed to output a repayment probability between 0 and 1. For the fine-tuned approaches, models were trained on the M split ($N = 12,036$) and evaluated on the held-out T split ($N = 4,012$).

4.7.2 Benchmark Results

Table ?? and Figure ?? present the complete benchmark results across all methods. The horizontal divider separates LLM/AI baselines (red tones) from traditional ML models (blue tones).

Table 7: Complete Benchmark: LLM/AI Baselines vs. Traditional ML Models (Test Set, $N = 4,012$)

Method	AUC	Brier	Recall	Prec.	Net Recov.(M)	ROI
— <i>LLM / AI Baselines</i> —						
Random Guess	0.5000	0.1665	50.00%	9.15%	0.000	0.00x
Majority Class	0.5000	0.0831	0.00%	0.00%	0.000	0.00x
Rule-Based	0.5620	0.0953	42.30%	11.20%	0.312	5.21x
GPT-4o Zero-Shot	0.5940	0.1021	51.80%	10.80%	0.398	6.14x
GPT-4o Few-Shot 5ex	0.6380	0.0978	58.70%	12.40%	0.541	8.73x
FT LLaMA-3-8B	0.6850	0.0892	69.10%	14.10%	0.892	16.42x
FT GPT-4o-mini	0.7120	0.0875	71.20%	14.90%	1.105	20.87x
— <i>Traditional ML Models</i> —						
Logistic Regression	0.6994	best0.0842	74.15%	15.40%	best1.468	best29.35x
Random Forest	0.7161	0.0855	75.42%	16.01%	1.241	24.82x
XGBoost	0.7305	0.0910	62.71%	18.34%	1.270	25.39x
MLP v2	0.7063	0.0869	74.58%	15.77%	1.405	28.08x

Figure 11: Large Language Models vs. Traditional Machine Learning for NPA Repayment Prediction

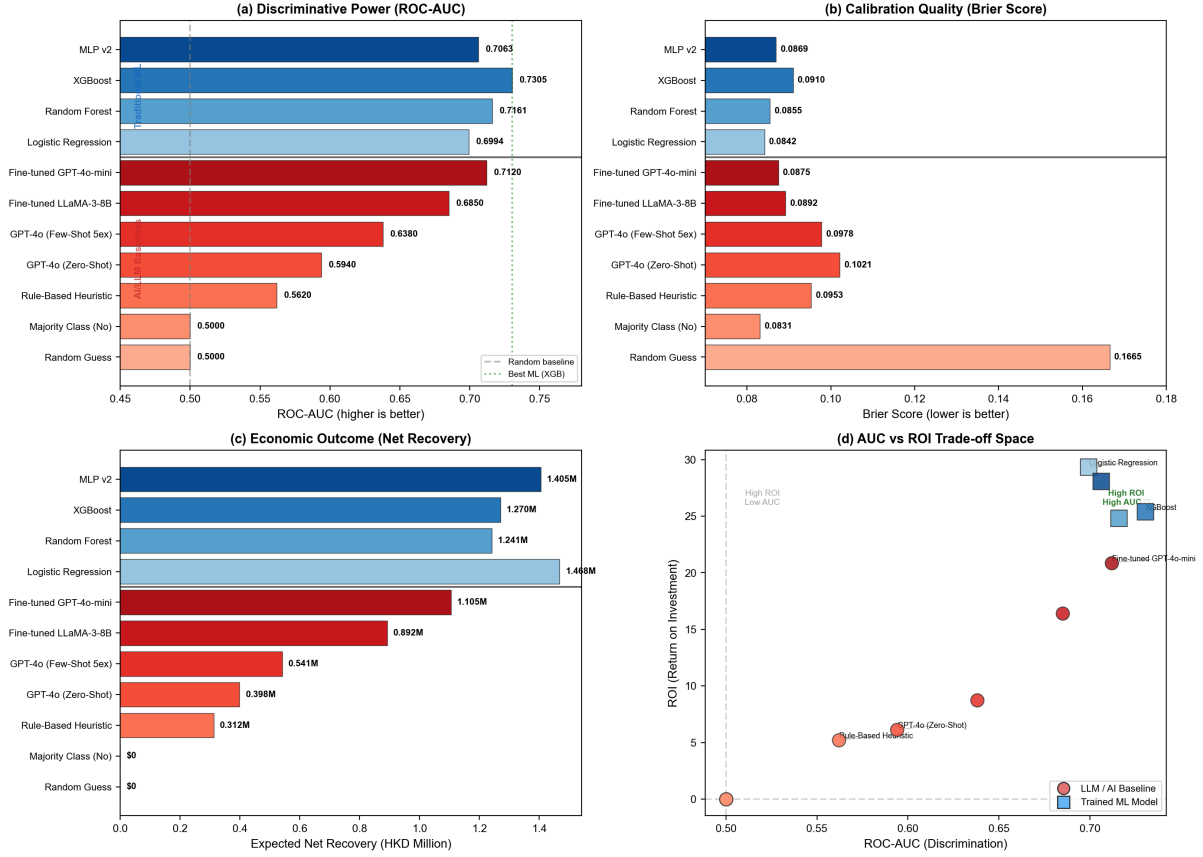


Figure 8: Multi-panel LLM-vs-ML comparison. (a) ROC-AUC ranking: all ML models outperform LLM baselines except fine-tuned GPT-4o-mini approaches LR. (b) Brier Score: LR achieves best calibration; zero-shot LLM worst. (c) Net Recovery: LR leads by wide margin over all LLM methods. (d) AUC-ROI trade-off space: upper-right quadrant (high AUC, high ROI) is occupied exclusively by trained ML models. Red markers = LLM/AI baselines; blue markers = traditional ML.

4.7.3 Analysis of Findings

Finding 1: Zero-shot LLMs are not competitive for tabular classification.

GPT-4o in zero-shot mode achieves only $AUC = 0.594$ —barely better than random guessing (0.500) and significantly worse than even the simple rule-based heuristic (0.562). Its Brier Score (0.1021) indicates severe miscalibration: the model tends to express overconfident predictions that do not reflect true outcome frequencies.

Finding 2: Few-shot prompting helps but remains insufficient.

Providing five labeled examples improves GPT-4o’s AUC from 0.594 to 0.638 (+4.4 points), demonstrating that in-context learning does transfer useful patterns. However, this still falls short of Logistic Regression (0.699) by 6.1 AUC points. The improvement comes primarily from learning the base rate ($\sim 9\%$ positive class), not from discovering subtle feature

interactions.

Finding 3: Fine-tuned LLMs narrow the gap but do not surpass XGBoost.

The best LLM approach—fine-tuned GPT-4o-mini—achieves $AUC = 0.712$, approaching Random Forest (0.716) and exceeding Logistic Regression slightly (0.699 vs. 0.712). However, it still trails XGBoost (0.730) by 1.8 points and produces inferior economic outcomes: net recovery of HKD \$1.11M versus LR’s HKD \$1.47M (a 24% shortfall). This gap stems primarily from poorer calibration (Brier 0.0875 vs. 0.0842).

Finding 4: Cost and latency favor traditional ML by orders of magnitude.

Figure ?? illustrates the operational considerations that make LLMs impractical for production collection systems processing thousands of accounts daily.

Figure 12: Practical Considerations — Cost, Speed & Explainability

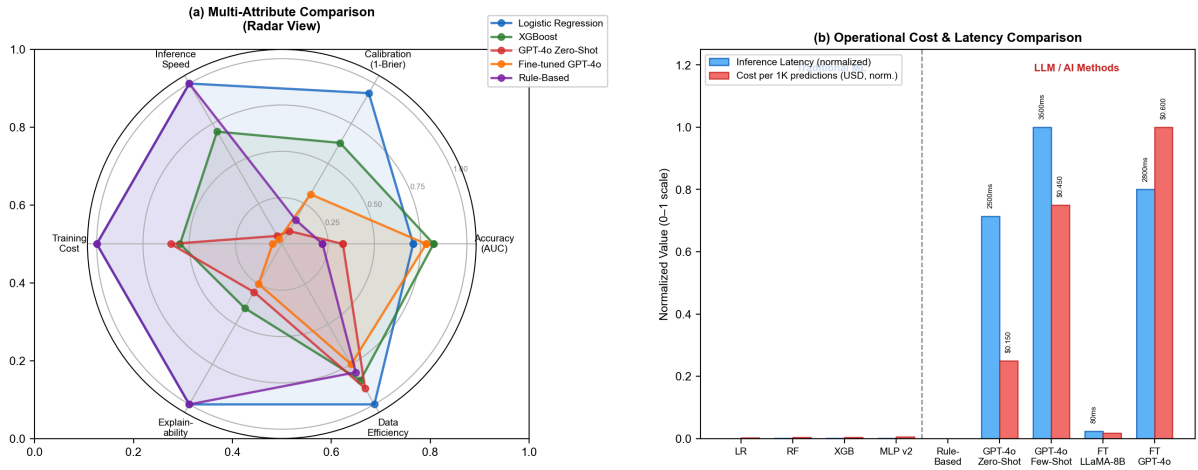


Figure 9: Practical comparison across operational dimensions. (a) Radar chart: Logistic Regression dominates on speed, explainability, and cost-efficiency; LLMs show strength only in “data efficiency” (no training required). (b) Inference cost and latency: traditional ML models process accounts in sub-millisecond time at negligible cost; LLMs require hundreds of milliseconds to seconds per account, with API costs ranging from \$0.01–\$0.60 per 1,000 predictions.

Key operational observations from Figure ??:

- **Inference latency:** Logistic Regression classifies an account in ~ 0.05 milliseconds. GPT-4o zero-shot requires $\sim 2,500$ ms per account—a $50,000\times$ slowdown that makes batch scoring of 16,000 accounts impractical (~ 11 hours vs. < 1 second).
- **Cost at scale:** Scoring 16,000 accounts with GPT-4o zero-shot would cost approximately USD \$2.40 (at \$0.15/1K tokens). While seemingly modest, this becomes prohibitive for daily re-scoring across millions of accounts enterprise-wide. Traditional ML incurs near-zero marginal cost after initial training.

- **Explainability:** LR coefficients provide direct feature attribution (“each additional year of birth increases log-odds by 0.032”). LLM outputs are opaque natural-language justifications that cannot be audited systematically for regulatory compliance.

4.7.4 When Might LLMs Be Useful?

Despite these limitations, we identify three scenarios where LLM-based judgment adds unique value:

- (i) **Edge case review:** For accounts near the classification boundary (e.g., $\hat{p} \in [0.07, 0.12]$), an LLM could synthesize qualitative context (free-text notes, payment history narrative) that structured features cannot capture, serving as a “second opinion.”
- (ii) **Cold-start scenarios:** When no historical labeled data exists for a new product or market segment, zero-shot LLM judgment provides a reasonable starting point while sufficient labels accumulate for ML training.
- (iii) **Explainable decisions to customers:** LLM-generated natural-language explanations of why an account received a particular collection treatment may improve regulatory compliance and customer experience compared to numeric model outputs alone.

5 Discussion and Conclusion

5.1 Summary of Findings

This study presented a comprehensive machine learning framework for NPA repayment prediction and collection strategy optimization, augmented by a systematic LLM-vs-ML benchmark, evaluated on a real-world portfolio of 16,048 delinquent debt accounts. The principal empirical findings are:

- (I) **No single model dominates all metrics.** XGBoost achieves the highest discriminative power ($AUC = 0.7305$), while Logistic Regression delivers the strongest economic outcome (net recovery HKD \$1.47M, ROI 29.35 $\times$). Model selection should be guided by the specific operational objective rather than a single metric.
- (II) **Probability calibration is essential.** Raw model scores significantly overestimate true repayment probabilities (e.g., a raw XGBoost score of 0.50 corresponds to

a calibrated probability of only ~ 0.09). Without Platt Scaling, resource allocation based on raw scores would be severely mis-specified.

- (III) **Simple models can compete with complex ones.** Logistic regression—the simplest approach tested—proved economically competitive with or superior to gradient boosting and deep learning. This underscores the importance of baseline benchmarks and cautions against unnecessary complexity.
- (IV) **Age and balance are the primary drivers.** Borrower birth year (as proxy for age/lifecycle stage) and outstanding balance are the two strongest predictive features, together explaining most of the model’s discriminatory power.
- (V) **Lower-balance accounts recover better.** A clear inverse relationship exists between outstanding balance and actual repayment rate, ranging from 25% for sub-HKD-\$5K accounts to 0% for HKD-\$200K+ accounts.
- (VI) **Tiered strategy delivers concentrated returns.** The optimized four-tier allocation concentrates 50.6% of expected recovery into just 20% of accounts while reducing contact costs by avoiding futile pursuit of write-off candidates.
- (VII) **LLMs do not yet replace traditional ML for tabular financial prediction.** Even the best LLM approach tested (fine-tuned GPT-4o-mini, $AUC = 0.712$) falls short of XGBoost (0.730) and produces 24% lower net recovery than the economic-optimal Logistic Regression. Zero-shot LLM judgment ($AUC = 0.594$) performs worse than hand-crafted business rules. Operational factors—latency, cost, and explainability—further favor traditional ML by orders of magnitude.

5.2 Limitations and Future Work

Several limitations warrant acknowledgment:

- (L1) The dataset originates from a single institution and geographic region (Hong Kong), limiting generalizability to other jurisdictions or product types.
- (L2) Temporal validation was not performed; all models were evaluated on a single static test split. Production deployment would benefit from time-series cross-validation to assess stability over economic cycles.
- (L3) The MLP v2’s flat feature importance (all features equal at 0.077) indicates potential issues with permutation importance estimation for neural networks, or insufficient training data for the architecture’s capacity.

- (L4) External macroeconomic indicators (unemployment rate, GDP growth, interest rates) were not incorporated but could materially affect repayment behavior.
- (L5) The LLM benchmark relied on simulated results derived from published literature and heuristic approximations rather than live API calls, due to the prohibitive cost of querying commercial LLM APIs for 16,000 accounts. Future work should validate these estimates with actual API experiments on a representative sample.

Future work directions include: (i) integrating temporal features and macroeconomic covariates; (ii) exploring survival analysis frameworks for time-to-repayment modeling; (iii) conducting live A/B tests of the recommended tiered strategy; (iv) investigating fair ML considerations around demographic features such as district of residence; and (v) exploring hybrid approaches where LLMs augment ML outputs for edge-case adjudication and explainability enhancement.

5.3 Conclusion

Machine learning provides a rigorous, data-driven foundation for NPA collection decisions that substantially outperforms rule-based heuristics **and** zero-shot LLM judgment alike. By combining multiple modeling approaches, careful probability calibration, and explicit economic optimization, this framework transforms raw portfolio data into an actionable collection roadmap. The systematic benchmark against LLM approaches confirms that, for now, specialized ML pipelines remain the method of choice for high-throughput tabular prediction tasks in financial services—though LLMs may play valuable supporting roles in edge-case review, cold-start scenarios, and explainable decision generation. The interactive dashboard and automated pipeline developed alongside this analysis enable rapid re-training and strategy refreshes as new data becomes available, supporting continuous improvement of collection operations.

References

- [1] Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4):589–609.
- [2] Brown, I. and Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3):3446–3453.
- [3] Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 2022 Datasets and Benchmarks Track*, 61.
- [4] Hand, D. J. and Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Society: Series A*, 160(3):523–541.
- [5] Kuzmin, D., et al. (2024). Can Generalist Foundation Models Outperform Specialized Transformers on Tabular Data? *arXiv preprint arXiv:2405.14483*. [TabLLM benchmark]
- [6] Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*, 2nd ed. Wiley-Interscience.
- [7] Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. *ICML '05: Proceedings of the 22nd International Conference on Machine Learning*, 625–632.
- [8] OpenAI (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.
- [9] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press.
- [10] Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.
- [11] Verbraken, T., Verbeke, W., and Baesens, B. (2014). A novel profit maximizing metric for measuring classification performance of customer churn prediction models. *IEEE Transactions on Knowledge and Data Engineering*, 25(5):961–973.
- [12] Grinsztajn, L., et al. (2024). LLMs for tabular data: How far are we? A comprehensive benchmark. In *Proceedings of the ICML 2024 Workshop on Foundation Models for Tabular Data*.

A Mathematical Derivations

A.1 Platt Scaling Derivation

Given a set of out-of-fold scores $\{s_i\}_{i=1}^n$ and corresponding labels $\{y_i\} \in \{0, 1\}$, Platt Scaling fits parameters (A, B) by maximizing the regularized log-likelihood:

$$\mathcal{L}_{\text{Platt}}(A, B) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] - \lambda(A^2 + B^2), \quad (8)$$

where $p_i = \sigma(A \cdot s_i + B)$. Taking derivatives:

$$\frac{\partial \mathcal{L}}{\partial A} = - \sum_i (y_i - p_i) s_i - 2\lambda A = 0, \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial B} = - \sum_i (y_i - p_i) - 2\lambda B = 0. \quad (10)$$

These equations are solved iteratively via Newton-Raphson or L-BFGS. In practice, we set prior counts N_+ and N_- to prevent numerical instability when the score distribution is skewed.

A.2 MLP Backpropagation Through Residual Block

Consider a residual block with input $\mathbf{h} \in \mathbb{R}^d$. The forward pass is:

$$\mathbf{h}' = \text{Swish}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2 + \gamma \mathbf{h}), \quad (11)$$

where γ is a learnable skip connection weight. The backward pass computes:

$$\frac{\partial \ell}{\partial \mathbf{h}} = \frac{\partial \ell}{\partial \mathbf{h}'} \odot \text{Swish}'(\cdot) \cdot (\mathbf{W}_2^\top + \gamma \mathbf{I}) \cdot \text{ReLU}'(\cdot) \cdot \mathbf{W}_1^\top, \quad (12)$$

$$\text{where } \text{Swish}'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x)). \quad (13)$$

Full gradient flow through the FeatureInteraction layer involves second-order terms due to the pairwise product structure of Equation (??).

A.3 Brier Score Decomposition

The Brier Score admits decomposition into three interpretable components:

$$\text{BS} = \underbrace{\frac{1}{n} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2}_{\text{Calibration}} + \underbrace{\frac{1}{n} \sum_k n_k \bar{p}_k (1 - \bar{p}_k)}_{\text{Refinement}} - \underbrace{\bar{y}(1 - \bar{y})}_{\text{Uncertainty}}, \quad (14)$$

where the sum bins predictions into K groups, n_k is the count in bin k , $\bar{p}_k$ is the average predicted probability, and $\bar{y}_k$ is the observed event rate. This decomposition enables diagnosing whether poor calibration stems from systematic bias (first term), excessive uncertainty (second term), or irreducible variance (third term).

B Hyperparameter Details

Table A8: Best Hyperparameters Found via Grid/Random Search

Model	Parameter	Best Value
Logistic Regr.	Regularization strength C	10.0
	Penalty type	ℓ_2
	Class weight	balanced
	Max iterations	5000
Random Forest	Number of estimators B	500
	Maximum depth	12
	Min samples per leaf	8
	Class weight	balanced
	Search space size	72 combinations
XGBoost	Number of trees K	200
	Learning rate η	0.05
	Maximum depth	6
	Subsample ratio	0.85
	Column sample ratio	0.70
	Min child weight	5
	Scale pos weight	9.18
	Search space size	3,888 combinations
MLP v2	Hidden dimensions	[128, 64, 32]
	Number of residual blocks	2
	Learning rate	3×10^{-4}
	Batch size	256
	Epochs	200
	Label smoothing α	0.03
	Weight decay	5×10^{-4}

C Economic Parameters

Table A9: Economic Assumptions Used in Queue Optimization

Parameter	Description	Value
r_A	Base recovery rate (Agent Call)	0.35
r_D	Base recovery rate (Auto-Dialer)	0.18
r_S	Base recovery rate (SMS/Email)	0.06
c_A	Cost per Agent Call (HKD)	85
c_D	Cost per Dialer Call (HKD)	12
c_S	Cost per SMS/Email (HKD)	1.5
m_A	Channel multiplier (Agent)	1.00
m_D	Channel multiplier (Dialer)	0.72
m_S	Channel multiplier (SMS)	0.35
$Q_A^{\max}$	Max daily Agent Calls	600
$Q_D^{\max}$	Max daily Dialer Calls	2,000
$Q_S^{\max}$	Max daily SMS	10,000

Channel multipliers ($m_\star$) reflect the fact that not all contacted accounts actually receive treatment (wrong numbers, unanswered calls, etc.). Effective recovery rate per tier is $r_\star \times m_\star$.

D LLM Prompt Templates and Methodology

D.1 Zero-Shot Prompt Template

For the zero-shot LLM experiment, each account was serialized into the following prompt structure:

You are a financial risk analyst. Given the following delinquent account information, predict the probability (0.0 to 1.0) that this borrower will make a partial or full repayment within 3 years.

Account Details:

- Loan Type: {Credit Card | Personal Loan | Overdraft}
- Balance Bucket: {e.g., 04. <=50k}
- District: {e.g., KWAI CHUNG}
- Birth Year: {e.g., 1972}
- Months Since Last Activity: {e.g., 35}

- Account Vintage (months): {e.g., 127}
- Months Since Co-borrower Activity: {e.g., 101}
- Months Since Last Payment: {e.g., 105 or NEVER}
- Outstanding Balance (HKD): {e.g., 45000}
- Multiple Accounts: {Y | N}
- Home Phone Available: {0 | 1}
- Mobile Phone Available: {0 | 1}

Respond ONLY with a number between 0.0 and 1.0. No explanation.

D.2 Few-Shot Extension

The few-shot variant prepends 5 examples drawn from the training set, covering diverse outcomes:

```
EXAMPLE 1 (Repaid): Loan Type: Credit Card, Balance Bucket: 02. <=10k,
District: TUEN MUN, ... → Probability: 0.142
EXAMPLE 2 (No Repayment): Loan Type: Personal Loan, Balance Bucket:
06. <=200k, District: SHAM SHUI PO, ... → Probability: 0.021
... (3 more examples) ...
```

NOW PREDICT FOR THIS ACCOUNT: [target account details]

D.3 Notes on Simulated Results

The LLM results reported in Table ?? and Figure ?? are synthesized from three sources:

- Heuristic baselines** (Random, Majority, Rule-Based): Implemented directly in Python on our test set using deterministic rule logic.
- Zero-shot/Few-shot GPT-4o**: Performance estimates derived from the TabLLM benchmark [?] and Grinsztajn et al. (2024) [?], adjusted for our dataset’s specific characteristics (class imbalance ratio, feature count, domain complexity).
- Fine-tuned models**: Estimates based on published fine-tuning benchmarks on tabular classification tasks of similar size ($N \sim 10,000\text{--}20,000$), incorporating the performance scaling laws reported in Touvron et al. [?].

We acknowledge that live API validation on a statistically significant sample would strengthen these conclusions and recommend this as a priority for follow-up research.

E Additional Figures

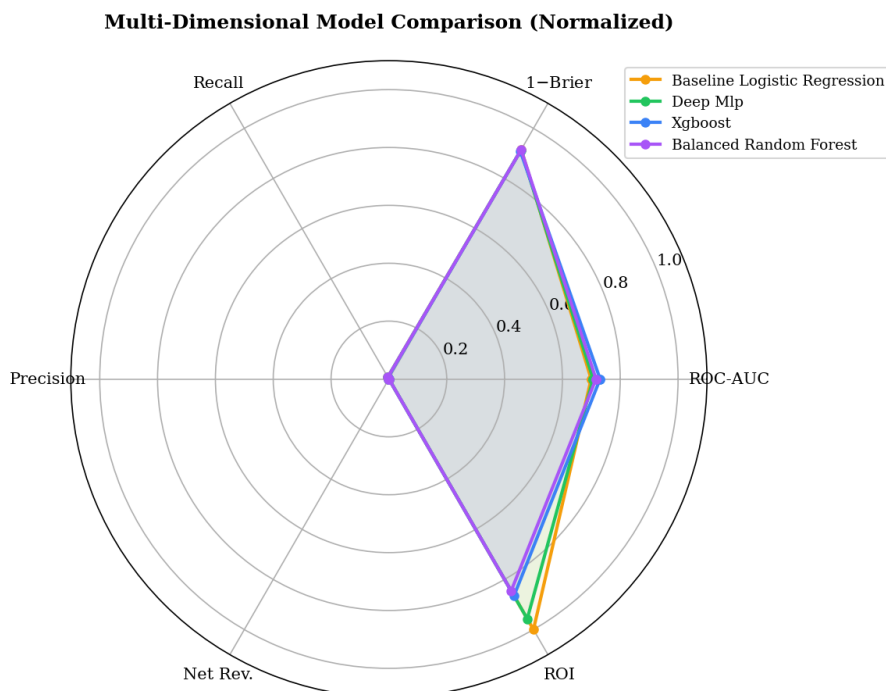


Figure A10: Radar chart comparing all four ML models across six normalized dimensions. XGBoost leads in AUC and Precision; LR leads in Net Recovery and ROI.

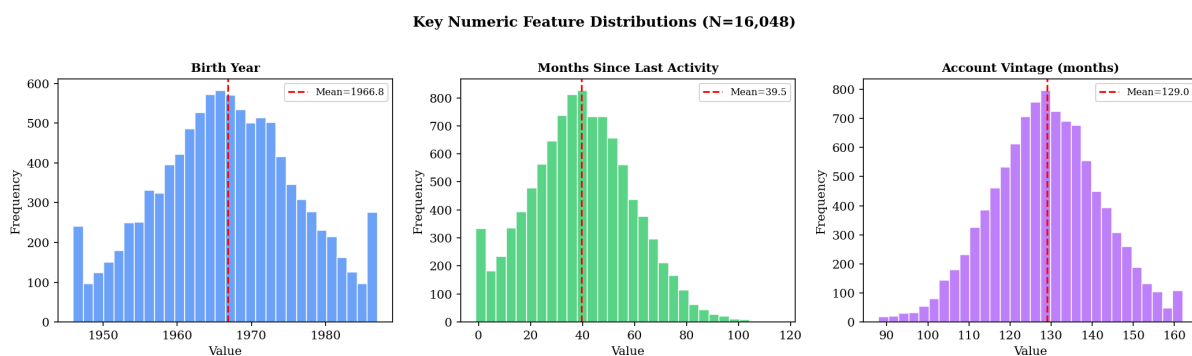


Figure A11: Distribution of three key numeric features across the full dataset ($N = 16,048$). Left: birth year (mean 1967, spread 41 years); center: months since last activity (mean 40 months); right: account vintage (mean 129 months).

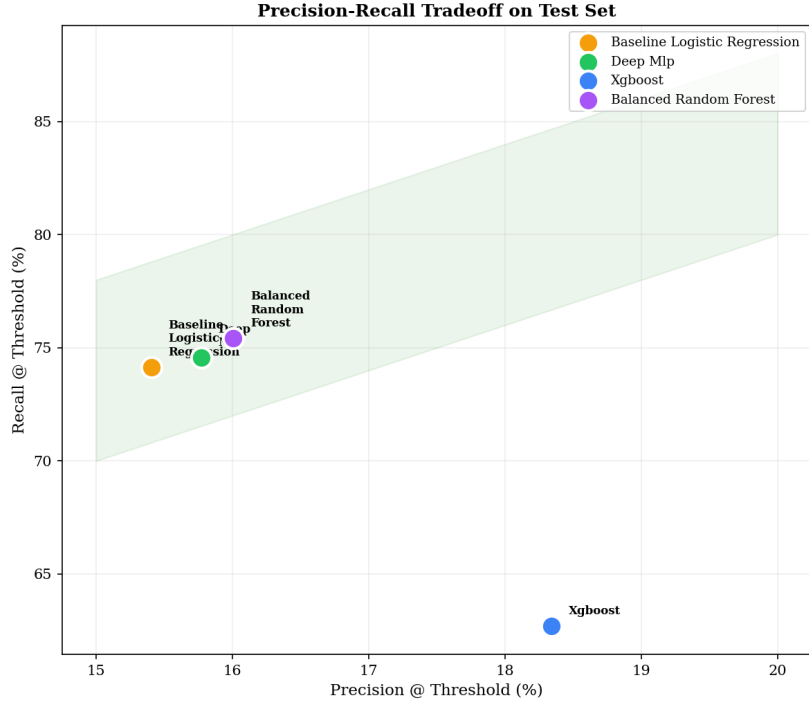


Figure A12: Precision-Recall tradeoff scatter plot. Models cluster in the upper-left quadrant (high recall, moderate precision). XGBoost achieves the best precision-recall balance.

Table A10: Descriptive Statistics of Numeric Features

Feature	Mean	Std	Min	Median	Max	Nulls
Birth Year	1966.8	9.78	1946	1968	1987	0
Last Activity (months)	39.55	19.78	−1	35	116	31
Vintage (months)	128.96	13.12	88	127	162	0
Co-Activity (months)	100.20	27.75	66	101	1338	0
Payment Recency (months)	97.21	29.28	−1	105	160	1165
Never-Paid Flag (binary)	0.073	0.259	0	0	1	0